K Nearest Neighbor

Fraida Fund

Contents

In this lecture . . . 2
Parametric vs. non-parametric models . . . 3
- So far... . . . 3
- Parametric models . . . 3
- Non-parametric models . . . 3
- Linear regression: a weighted sum of training labels . . . 3

Nearest neighbor . . . 4
- 1-NN . . . 4
- 1-NN - runtime . . . 4
- 1NN - approximation as a step function . . . 5
- 1NN - decision boundaries . . . 5
- K nearest neighbors . . . 6
- KNN for classification . . . 6
- KNN for regression . . . 6

What's left to decide? . . . 7
- What value of K? Illustration (1NN) . . . 7
- What value of K? Illustration (2NN) . . . 7
- What value of K? Illustration (3NN) . . . 8
- What value of K? Illustration (9NN) . . . 8
- What value of K? (1) . . . 8
- What value of K? (2) . . . 9
- What distance measure? (1) . . . 10
- Distance measure - standardization (1) . . . 10
- Distance measure - standardization (2) . . . 10
- Distance measure - equal weighted features . . . 11
- Distance measure - perceptual distance . . . 11
- How to combine labels into prediction? . . . 12

Bias and variance of KNN . . . 12
- True function . . . 12
- Assumption of fixed training set . . . 12
- Expected loss . . . 13
- KNN output . . . 13
- Bias of KNN . . . 13
- Variance of KNN (1) . . . 14
- Variance of KNN (2) . . . 14
- Error of KNN . . . 14
- Bias variance tradeoff . . . 14

The Curse of Dimensionality . . . 15
- KNN in 1D . . . 15
- KNN in 2D . . . 15
- Density of samples decreases with dimensions . . . 16

Density of samples decreases with dimensions - general 16
Solutions to the curse (1) 16
Solutions to the curse (2) 16
Summary of NN method 17
NN learning 17
NN prediction 17
KNN as a weighted sum of training labels 17
The good and the bad (1) 17
The good and the bad (2) 17
The good and the bad (3) 17

Math prerequisites for this lecture: You should know about

- probabilities, conditional probabilities, expectation of a random variable
- norm of a vector (Section I, Chapter 3 in Boyd and Vandenberghe)
- complexity of algorithms and especially of vector and matrix operations (Appendix B in Boyd and Vandenberghe, also the complexity part of Section I, Chapter 1 and Section II, Chapter 5)

In this lecture

- Parametric vs. non-parametric models
- Nearest neighbor
- What's left to decide?
- Bias and variance of KNN
- The Curse of Dimensionality

Parametric vs. non-parametric models

So far...

All of our models have looked like

$$\hat{y} = f(x, w) = w_0 + w_1 x_1 + \cdots + w_d x_d$$

A model class is more flexible if $f(x, w)$ can represent more possible functions.

- Some possibilities for $f(x, w) = w_0 + w_1 x_1$
- More possibilities for $f(x, w) = w_0 + w_1 x_1 + w_2 x_2$
- Even more possibilities for $f(x, w) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$

But until now, we had to "know" how to add flexibility - for example, by adding interaction terms or other basis functions.

A way to get more flexible models is with a **non-parametric** approach, where we don't a priori impose the functional form or a fixed number of parameters that the model should learn.

Parametric models

- A particular model class is assumed (e.g. linear)
- Number of parameters fixed in advance

Note: even if you use K-fold CV to try different candidate models with different number of parameters, you are still defining each candidate model in advance.

Non-parametric models

- Minimal assumptions about model class
- Model structure determined by data

Linear regression: a weighted sum of training labels

We previously showed that for linear regression (one feature, no intercept), the prediction for a point x_0 can be written

$$\hat{y}_0 = \sum_{j=1}^{N} a_{0j}\, y_j, \quad a_{0j} = \frac{x_0 x_j}{\sum_{k=1}^{N} x_k^2}$$

i.e. a weighted sum of the y values in the training set. Note: with an intercept or multiple features, the weights have a more complicated form, but the same idea holds.

Same model, two expressions:

- In terms of the parameter $\widehat{w}$: the training data is *summarized* by a fixed number of parameters. After training, we can throw away the training set, and the model does not get bigger when N gets bigger.
- In terms of the training data (this slide): the prediction is a weighted sum of the training labels.

Linear regression is **parametric**: the number of parameters is fixed in advance, and doesn't depend on N. Everything we need at prediction time fits in $\widehat{w}$.

In a **non-parametric** model, the size of the representation is not fixed in advance - it grows with the data.

Nearest neighbor

- A kind of non-parametric model.
- Basic idea: Find labeled samples that are "similar" to the new sample, and use their labels to make prediction for the new sample.

We previously spoke about the inductive bias of the linear models - we assumed that the target variable can be modeled as a linear combination of features or transformed versions of features. What is the inductive bias here?

1-NN

- Given training data $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$
- And a new sample $\mathbf{x}_0$
- Find the sample in the training data $\mathbf{x}_{i'}$ with the least distance to $\mathbf{x}_0$.

$$i' = \underset{i=1,\ldots,N}{\text{argmin}}\ d(\mathbf{x}_i, \mathbf{x}_0)$$

- Let $\hat{y}_0 = y_{i'}$



Figure 1: The nearest neighbor to the orange test point is the one that has the smallest distance in the feature space.

1-NN - runtime

- Training: just store data
- Inference: need to
  - compute distance to each of N points
  - distance metric typically scales with d

The runtime for predicting *each* test point is $O(Nd)$. How does this compare to linear regression or logistic regression?

There, the training time could be long, but *inference* was only $O(d)$. We can tolerate a long training time more easily than a long inference time.

1NN - approximation as a step function

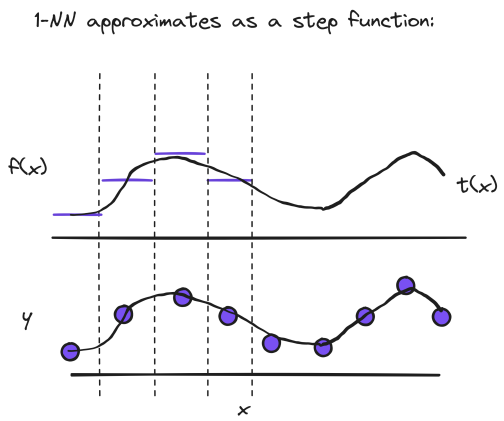


Figure 2: The nearest neighbor model approximates the true function using steps.

1NN - decision boundaries

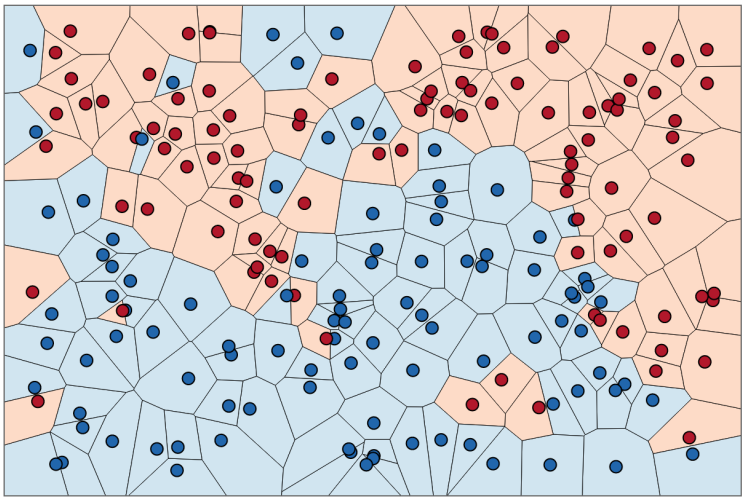


Figure 3: 1NN decision boundaries - Nearest neighborhoods for each point of the training data set.

Note that there will be zero error on training set (unless there are training data points that have identical feature values, but different labels).

K nearest neighbors

Instead of 1 closest sample, we find K: Let N_0 be the set of K training points that are closest to $\mathbf{x}_0$.

How do we use this set for (1) classification? (2) regression?



Figure 4: 3 nearest neighbors.

KNN for classification

Idea: Estimate conditional probability for a class as fraction of points among neighbors with the class label.

Remember: Let N_0 be the set of K training points that are closest to $\mathbf{x}_0$.

Then, we can estimate the per-class conditional probability given the sample x_0.

For each class $m \in M$:

$$P(y = m|\mathbf{x}_0) = \frac{1}{K} \sum_{(\mathbf{x}_i, y_i) \in N_0} I(y_i = m)$$

where $I(y_i = m)$ is 1 if $(\mathbf{x}_i, y_i) \in N_0$ is a member of class m, 0 otherwise.

- We can then select the class with the highest probability.
- Practically: select the most frequent class among the neighbors.

KNN for regression

Idea: Use the combined label of the K nearest neighbors. For example, we can take their mean:

$$\hat{y}_0 = \frac{1}{K} \sum_{(\mathbf{x}_i, y_i) \in N_0} y_i$$

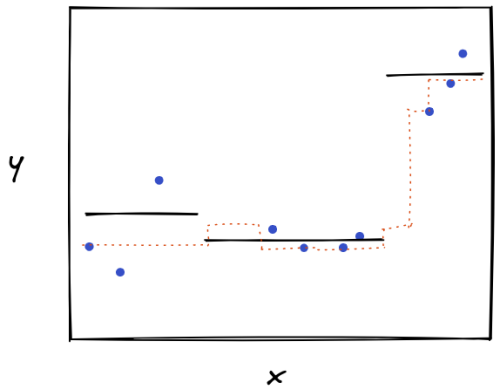


Figure 5: Example of a regression using median vote of the 3NN. The “true function” is shown in black, the orange dashed line shows the prediction of the regression model.

What's left to decide?

We're letting the data dictate the form of the solution, but we still need to make many model choices:

- What value of K?
- What distance measure?
- How to combine K labels into prediction?

Question: does larger K mean "more flexible" or "less flexible" model?

What value of K? Illustration (1NN)

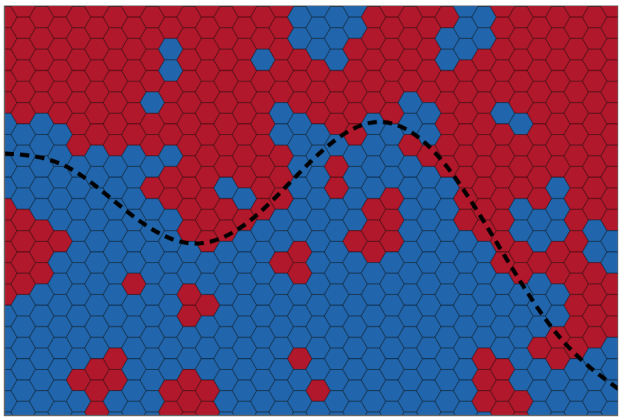


Figure 6: 1NN

What value of K? Illustration (2NN)

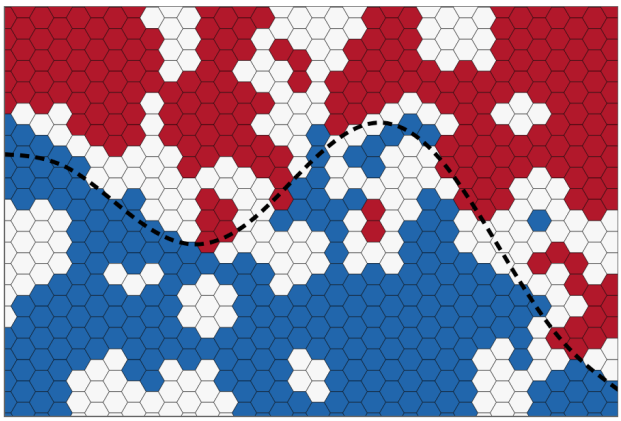


Figure 7: 2NN

What value of K? Illustration (3NN)

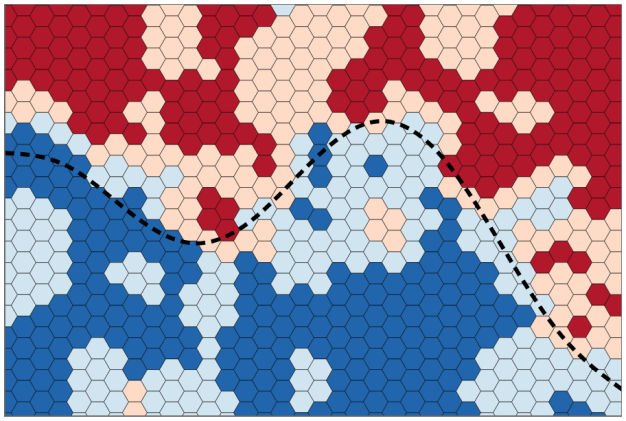


Figure 8: 3NN

What value of K? Illustration (9NN)

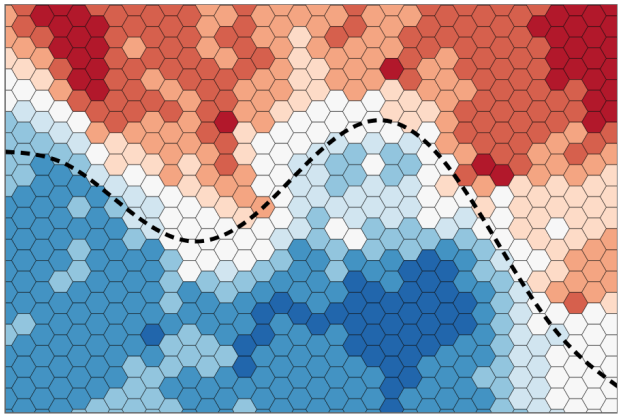


Figure 9: 9NN

What value of K? (1)

- In general: larger K, less complex model
- K can be selected by CV.
- Often cited “rule of thumb”: use $K = \sqrt{N}$

What value of K? (2)

- Alternative to fixed K: Radius-based neighbor learning.
- A fixed radius r is specified, can be selected by CV.
- Number of neighbors depends on local density of points.

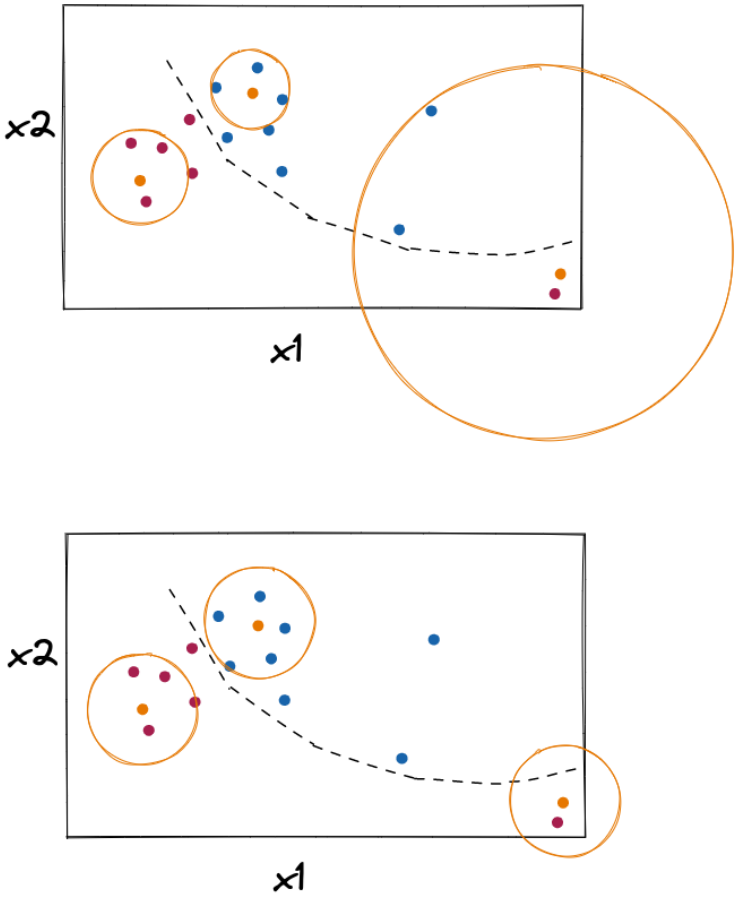


Figure 10: Comparison of KNN (top) and radius-based neighbor learning (bottom) when the density of training points in the feature space is uneven.

What distance measure? (1)

Some popular choices: for two vectors a_i, b_i,

- Euclidean (L2): $\sqrt{\sum_{i=1}^{d}(a_i - b_i)^2}$
- Manhattan (L1): $\sum_{i=1}^{d}|a_i - b_i|$

(L2 distance prefers many medium-sized disagreements to one big one.)

There are many more choices - for example, look at the distance metrics implemented in sklearn.

Problems with the basic distance metrics:

- When features have different scale/range, need to standardize
- KNN implicitly weights all features equally: this is a problem if you have features that are not relevant for the target variable!
- For images: pixel-wise distance doesn't necessarily equate to perceptual similarity

Distance measure - standardization (1)

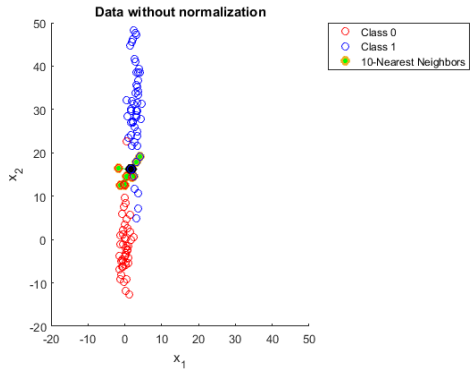


Figure 11: Without standardization, via https://stats.stackexchange.com/a/287439/. The x2 feature dominates the distance measure.

Distance measure - standardization (2)

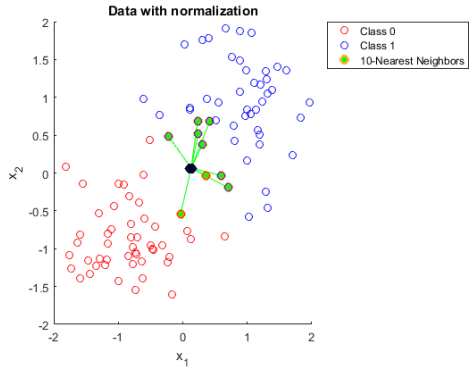


Figure 12: With standardization, via https://stats.stackexchange.com/a/287439/

Distance measure - equal weighted features

Suppose you are trying to predict a student's course grade using their previous GPA and how far they live from the NYU campus:

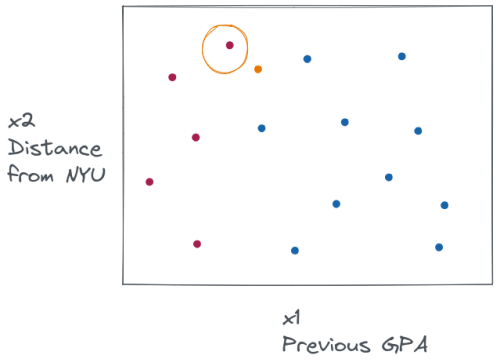


Figure 13: The training point circled in orange is the nearest neighbor, but it's not the most similar according to the only feature that matters (previous GPA). All features are weighted equally in the distance metric, regardless of their importance.

Alternative to equal weighted features: assign feature weights

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^{d} w_i (a_i - b_i)^2}$$

But then we need a way to learn feature weights!

With L1 regularization, we had a data-driven way to do feature selection. The nearest neighbor method doesn't have any "built-in" way to do feature weighting or feature selection as part of the training process, so we need to do it ourselves as part of the pre-processing steps.

We'll go back to this at the end.

Distance measure - perceptual distance

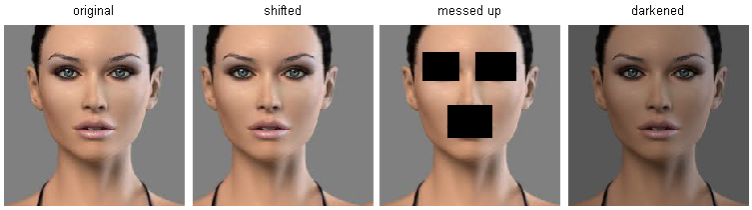


Figure 14: An original image (left) and three other images next to it that are all equally far away from it based on L2 pixel distance. Image via CS321n.

This is a little more difficult to overcome. In practice, KNN is often just not very useful for image data.

How to combine labels into prediction?

- **Basic voting**: use mode of neighbors for classification, mean or median for regression.
- **Distance-weighted**: weight of vote inversely proportional to distance from the query point. ("More similar" training points count more.)

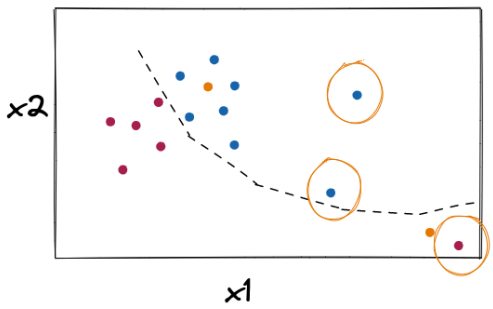


Figure 15: In this example, the red training point will get a bigger "vote" in the class label because it is closest to the test point. The point can be classified as red, even though 2 of the 3 neighbors are blue.

Bias and variance of KNN

True function

Suppose data has true relation

$$y = t(\mathbf{x}) + \epsilon, \quad \epsilon \sim N(0, \sigma_\epsilon^2)$$

and our model predicts $\hat{y} = f(\mathbf{x})$.

Assumption of fixed training set

For this derivation, we consider the expectation over:

- the test points
- the error ϵ
- the randomness in the y values in the training set!

We do not consider randomness in the x values - we assume a fixed training set.

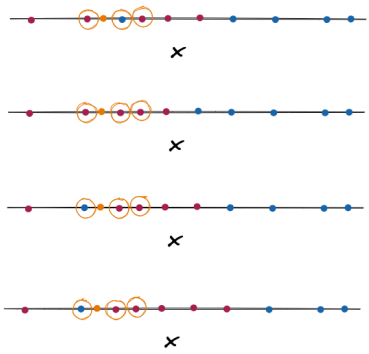


Figure 16: Assume the x values in the training set are fixed (and therefore, the neighbors of any given test point), but there is randomness in the y values.

Expected loss

We will use an L2 loss function, so that the expected error of the prediction $\hat{y}$ for a given test point $\mathbf{x}_t$ is:

$$\begin{aligned} MSE(\mathbf{x}_t) :=& E[(y-\hat{y})^2] \\ =& \left(t(\mathbf{x}_t) - E[f(\mathbf{x}_t)]\right)^2 + \\ & E[(f(\mathbf{x}_t) - E[f(\mathbf{x}_t)])^2] + \\ & \sigma_\epsilon^2 \end{aligned}$$

i.e. squared bias, variance, and irreducible error.

KNN output

The output of a KNN regression at the test point is

$$f(\mathbf{x}_t) = \frac{1}{K} \sum_{\ell \in K_x} t(\mathbf{x}_\ell) + \epsilon_\ell$$

where K_x is the set of K nearest neighbors of $\mathbf{x}_t$. (We assume that these neighbors are fixed.)

Bias of KNN

When we take expectation of bias over test samples:

$$\begin{aligned} Bias^2 &= \left(t(\mathbf{x}_t) - E[f(\mathbf{x}_t)]\right)^2 \\ &= \left(t(\mathbf{x}_t) - E\left(\frac{1}{K} \sum_{\ell \in K_x} t(\mathbf{x}_\ell) + \epsilon_\ell\right)\right)^2 \\ &= \left(t(\mathbf{x}_t) - \frac{1}{K} \sum_{\ell \in K_x} t(\mathbf{x}_\ell)\right)^2 \end{aligned}$$

The expectation is over the training sample draw - but note that the x values in the training samples are fixed! So the only randomness is in ϵ_ℓ.

Since the x values in the training samples are fixed, the $\frac{1}{K} \sum_{\ell \in K_x} t(\mathbf{x}_\ell)$ can come out of the expectation as a constant. Then $E[\epsilon_\ell] = 0$.

Variance of KNN (1)

$$
\begin{aligned}
Var(\hat{y}) &= E[(f(\mathbf{x}_t) - E[f(\mathbf{x}_t)])^2] \\
&= E\left[\left(f(\mathbf{x}_t) - \frac{1}{K}\sum_{\ell \in K_x} t(\mathbf{x}_\ell)\right)^2\right] \\
&= E\left[\left(\frac{1}{K}\sum_{\ell \in K_x} (t(\mathbf{x}_\ell) + \epsilon_\ell) - \frac{1}{K}\sum_{\ell \in K_x} t(\mathbf{x}_\ell)\right)^2\right] \\
&= E\left[\left(\frac{1}{K}\sum_{\ell \in K_x} \epsilon_\ell\right)^2\right]
\end{aligned}
$$

Variance of KNN (2)

$$
\begin{aligned}
&= E\left[\left(\frac{1}{K}\sum_{\ell \in K_x} \epsilon_\ell\right)^2\right] = \frac{1}{K^2} E\left[\left(\sum_{\ell \in K_x} \epsilon_\ell\right)^2\right] \\
&= \frac{1}{K^2} Var\left(\sum_{\ell \in K_x} \epsilon_\ell\right) = \frac{1}{K^2}\sum_{\ell \in K_x} Var\left(\epsilon_\ell\right) = \frac{K\sigma_\epsilon^2}{K^2} \\
&= \frac{\sigma_\epsilon^2}{K}
\end{aligned}
$$

Note: we use the fact that the ϵ terms are independent, so the variance of sum is equal to sum of variances.

Error of KNN

Then the expected error of KNN is

$$
\left(t(\mathbf{x}_t) - \frac{1}{K}\sum_{\ell \in K_x} t(\mathbf{x}_\ell)\right)^2 + \frac{\sigma_\epsilon^2}{K} + \sigma_\epsilon^2
$$

where K_x is the set of K nearest neighbors of $\mathbf{x}$.

Bias variance tradeoff

- Variance decreases with K
- Bias likely to increase with K, if function $t()$ is smooth.

Why does bias increase with K? For a smooth function, the few closest neighbors to the test point will have similar values, so average will be close to $t(\mathbf{x})$; as K increases, neighbors are further away, and average of neighbors moves away from $t(\mathbf{x})$.

You can think about the extreme case, where $K = N$ so you use the average of *all* of the training samples. This is equivalent to "prediction by mean".

The Curse of Dimensionality

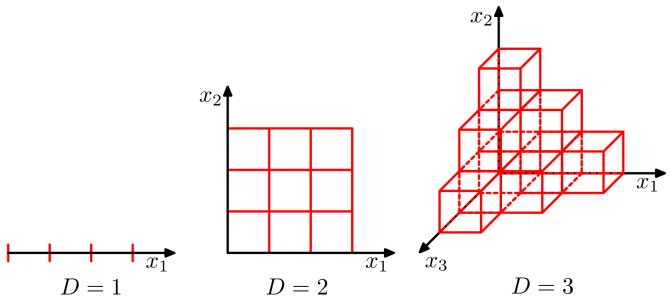


Figure 17: Feature space grows exponentially with dimension. From Bishop PRML, Fig. 1-21

KNN in 1D

- Consider a dataset $(x_1, y_1), \ldots, (x_N, y_N), N = 100$
- x is uniformly distributed in [0,1]
- On average, one data point is located every 1/100 units along 1D feature axis.
- To find 3NN, would expect to cover 3/100 of the feature axis.

KNN in 2D

- Now consider the same dataset with two features.
- Each feature is uniformly distributed in [0,1]
- To find 3NN, would expect to cover $0.03^{\frac{1}{2}}$ of the unit rectangle.

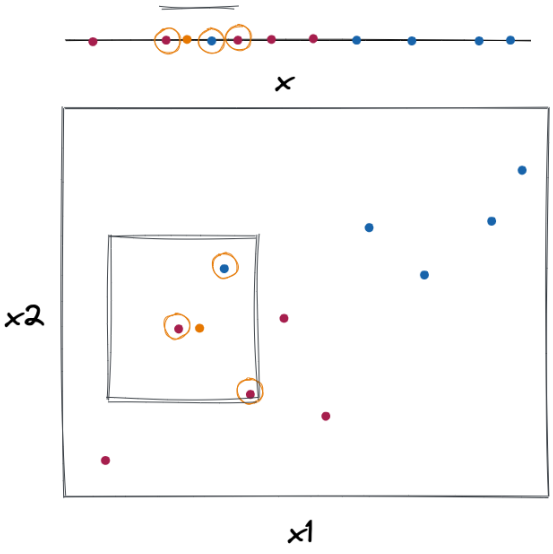


Figure 18: When d goes from 1 to 2, the density of the training points decreases and we need to cover more of the feature space to find the same number of neighbors.

Density of samples decreases with dimensions

To get 3NN,

- need to span 3% of the axis in 1D
- need to span 17% of each axis in 2D
- need to span 70% of each axis in 10D. At this point, the nearest neighbors are not much closer than the rest of the dataset.

Density of samples decreases with dimensions - general

The length of the smallest hyper-cube that contains all K-nearest neighbors of a test point:

$$\left(\frac{K}{N}\right)^{\frac{1}{d}}$$

for N samples with dimensionality d.

Solutions to the curse (1)

Add training data?

$$\left(\frac{K}{N}\right)^{\frac{1}{d}}$$

As number of dimensions increases linearly, number of training samples must increase exponentially to counter the “curse”.

Solutions to the curse (2)

Reduce d?

- Feature selection
- Dimensionality reduction: a type of unsupervised learning that *transforms* high-d data into lower-d data.

Summary of NN method

NN learning

Learning:

- Store training data
- Don't do anything else until you have a new point to classify

In practice, we will usually store training data in a data structure that makes it faster to compute nearest neighbors.

NN prediction

Prediction:

- Find nearest neighbors using distance metric
- Classification: predict most frequently occurring class among nearest neighbors
- Regression: predict mean value of nearest neighbors

KNN as a weighted sum of training labels

For KNN regression, the prediction for a point $\mathbf{x}_0$ is

$$\hat{y}_0 = \sum_{j=1}^{N} a_{0j}\, y_j, \quad a_{0j} = \begin{cases} \frac{1}{K} & \mathbf{x}_j \in N_0 \\ 0 & \text{otherwise} \end{cases}$$

Compare to linear regression, where $a_{0j} = \frac{x_0 x_j}{\sum_{k=1}^{N} x_k^2}$:

- Linear regression: the weights are global.
- KNN: the weights are local. Only the K nearest training points matter.

The weights for linear regression can be summarized in $\hat{w}$, so we can throw away the training data. The weights for KNN depend on where $\mathbf{x}_0$ is relative to the training points, so we have to keep all of the training data - the size of the representation grows with N.

The good and the bad (1)

Good:

- Good interpretability
- Fast "learning" (*memory-based*)
- Works well in low dimensions for complex decision surfaces

The good and the bad (2)

Neutral:

- Assumes similar inputs have similar outputs

The good and the bad (3)

Bad:

- Slow prediction (especially with large N)
- Curse of dimensionality